Parallel and Distributed Computing

Assignment 5

Submitted to: Dr Saif Nalband
Submitted by: Anushka Gupta
Roll No:102303358
Course: UCS645

Introduction

This assignment focuses on understanding the difference between blocking and non-blocking communication in MPI, the importance of collective operations, and dynamic load balancing using the master-slave pattern. Five programs were developed and executed on Google Colab using MPICH. Performance was measured using `MPI_Wtime()` for speedup and efficiency analysis.

All programs were compiled using the provided `Makefile` and tested with 1, 4, and 8 processes.

Question 1: DAXPY Loop

> **Question 1.** DAXPY Loop D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^16 each, P stands for Plus. The operation to be completed in one iteration is X[i] = a*X[i] + Y[i]. Implement an MPI program to complete the DAXPY operation. Measure the speedup of the MPI implementation compared to a uniprocessor implementation. The double MPI Wtime() function is the equivalent of the double omp get wtime().

Objective: Implement parallel DAXPY operation (`X[i] = a * X[i] + Y[i]`) and measure speedup compared to serial execution.

```
%%writefile q1_daxpy.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N (1<<20)   // 65536 as per assignment

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double a = 3.14;
    double *X = (double*)malloc(N * sizeof(double));
    double *Y = (double*)malloc(N * sizeof(double));

    // Initialize on rank 0 and scatter
    if (rank == 0) {
        for (int i = 0; i < N; i++) {
            X[i] = i * 0.1;
            Y[i] = i * 0.2;
        }
    }

    int local_n = N / size;
    double *local_X = (double*)malloc(local_n * sizeof(double));
    double *local_Y = (double*)malloc(local_n * sizeof(double));

    MPI_Scatter(X, local_n, MPI_DOUBLE, local_X, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Scatter(Y, local_n, MPI_DOUBLE, local_Y, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

  MPI_Barrier(MPI_COMM_WORLD);   // sync all processes
double start = MPI_Wtime();

// DAXPY computation
for (int i = 0; i < local_n; i++) {
    local_X[i] = a * local_X[i] + local_Y[i];
}

MPI_Barrier(MPI_COMM_WORLD);   // ensure all finished
double end = MPI_Wtime();

    // Gather back (just for verification)
    MPI_Gather(local_X, local_n, MPI_DOUBLE, X, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("DAXPY completed successfully!\n");
        printf("Time taken with %d processes: %f seconds\n", size, end - start);
        printf("Speedup can be measured by comparing np=1 vs np=4/8\n");
    }

    free(local_X); free(local_Y); free(X); free(Y);
    MPI_Finalize();
    return 0;
}
```

```
Overwriting q1_daxpy.c
```

Results:

```
!mpicc -o q1_daxpy q1_daxpy.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./q1_daxpy

DAXPY completed successfully!
Time taken with 4 processes: 0.005619 seconds
Speedup can be measured by comparing np=1 vs np=4/8
```

```
!mpirun --allow-run-as-root --oversubscribe -np 1 ./q1_daxpy
!mpirun --allow-run-as-root --oversubscribe -np 2 ./q1_daxpy
!mpirun --allow-run-as-root --oversubscribe -np 4 ./q1_daxpy
!mpirun --allow-run-as-root --oversubscribe -np 8 ./q1_daxpy

DAXPY completed successfully!
Time taken with 1 processes: 0.003559 seconds
Speedup can be measured by comparing np=1 vs np=4/8
DAXPY completed successfully!
Time taken with 2 processes: 0.002924 seconds
Speedup can be measured by comparing np=1 vs np=4/8
DAXPY completed successfully!
Time taken with 4 processes: 0.004492 seconds
Speedup can be measured by comparing np=1 vs np=4/8
DAXPY completed successfully!
Time taken with 8 processes: 0.004005 seconds
Speedup can be measured by comparing np=1 vs np=4/8
```

Key Implementation:
Vector size N = 65536
Data is distributed among processes using MPI_Scatter
Each process performs local DAXPY computation: X[i] = a * X[i] + Y[i]
Final results are collected using MPI_Gather
Execution time is measured using MPI_Wtime()

Performance Results:
For 1 process → Time = 0.003559 seconds, Speedup = 1.00, Efficiency = 100%
For 2 processes → Time = 0.002924 seconds, Speedup = 1.22, Efficiency = 61%
For 4 processes → Time = 0.004492 seconds, Speedup = 0.79, Efficiency = 20%
For 8 processes → Time = 0.004005 seconds, Speedup = 0.89, Efficiency = 11%

Observation:
A slight performance improvement is observed when increasing from 1 to 2 processes.
Beyond that, execution time increases instead of decreasing.
This indicates that communication and synchronization overhead dominate the computation.
As the number of processes increases, the workload per process becomes too small to benefit from parallelization.

Analysis:
The speedup is not linear due to communication overhead from MPI_Scatter and

MPI_Gather operations.
Process synchronization also contributes to delays.
The problem size is relatively small, which limits the effectiveness of parallel execution.
Efficiency decreases with increasing number of processes, indicating poor scalability.

Conclusion:
The MPI implementation of the DAXPY operation is correct and successfully executed.
Parallelization provides limited performance improvement for small input sizes.
Increasing the number of processes beyond a certain point leads to performance degradation.
Better results can be achieved by increasing the vector size so that computation outweighs communication overhead.

Question 2: The Broadcast Race (Linear vs. Tree Communication)

Objective: Understand the underlying algorithmic efficiency of MPI's built-in collective operations compared to manual point-to-point communication.

The Premise: When a novice programmer needs to send the same massive array from Rank 0 to 15 other processes, their first instinct is often to write a for loop of MPI_Send calls. However, MPI_Bcast is heavily optimized and often uses a "tree-based" distribution (Rank 0 sends to 1, then 0 and 1 send to 2 and 3, etc.), which is mathematically much faster.

The Task (Programming): Write a C program that allocates a massive array of 10 million double values (approx. 80 MB).

1. **Part A (MyBcast):** Write a custom function where Rank 0 uses a for loop and standard MPI_Send to transmit this array to all other ranks. The other ranks will use MPI_Recv to catch it. Use MPI_Wtime() to measure exactly how long this takes.
2. **Part B (MPI_Bcast):** Reset the array, and do the exact same data transfer using the built-in MPI_Bcast function. Measure the time.

The Experiment (Analysis & Report):

1. Run your program on a cluster (or multicore machine) using **2, 4, 8, and 16 processes.**
2. Record the completion times for both "MyBcast" and "MPI_Bcast" for each process count.
3. **Questions to Answer:**

3. **Questions to Answer:**

   - Create a line graph comparing the execution time of both methods as the number of processes increases.

   - Does the time taken by your for-loop broadcast scale linearly? Why?
   - How does the execution time of MPI_Bcast change as you add more processes? Explain the theoretical time complexity difference between the two approaches.

Code:

```
%%writefile q2_broadcast.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define ARRAY_SIZE 10000000   // 10 million doubles (~80 MB)

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double *data = (double*)malloc(ARRAY_SIZE * sizeof(double));
    double start, end;

    if (rank == 0) printf("=== Running with %d processes ===\n", size);

    // ------------------ Part A: MyBcast (linear loop) ------------------
    if (rank == 0) {
        for (int i = 0; i < ARRAY_SIZE; i++) data[i] = i * 0.1;
        start = MPI_Wtime();
        for (int i = 1; i < size; i++) {
            MPI_Send(data, ARRAY_SIZE, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
        }
    } else {
        MPI_Recv(data, ARRAY_SIZE, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    MPI_Barrier(MPI_COMM_WORLD);
    if (rank == 0) {
        end = MPI_Wtime();
        printf("MyBcast (linear) time: %f seconds\n", end - start);
    }

    // ------------------ Part B: MPI_Bcast ------------------
    if (rank == 0) {
        for (int i = 0; i < ARRAY_SIZE; i++) data[i] = i * 0.1;
        start = MPI_Wtime();
    }
```

```c
    // ------------------ Part B: MPI_Bcast ------------------
    if (rank == 0) {
        for (int i = 0; i < ARRAY_SIZE; i++) data[i] = i * 0.1;
        start = MPI_Wtime();
    }
    MPI_Bcast(data, ARRAY_SIZE, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Barrier(MPI_COMM_WORLD);
    if (rank == 0) {
        end = MPI_Wtime();
        printf("MPI_Bcast time: %f seconds\n", end - start);
    }

    free(data);
    MPI_Finalize();
    return 0;
}
```

```
Writing q2_broadcast.c
```

Results :

```
!mpicc -o q2_broadcast q2_broadcast.c
```

```
!mpirun --allow-run-as-root --oversubscribe -np 2 ./q2_broadcast
!mpirun --allow-run-as-root --oversubscribe -np 4 ./q2_broadcast
!mpirun --allow-run-as-root --oversubscribe -np 8 ./q2_broadcast
!mpirun --allow-run-as-root --oversubscribe -np 16 ./q2_broadcast
```

```
=== Running with 2 processes ===
MyBcast (linear) time: 0.081504 seconds
MPI_Bcast time: 0.036116 seconds
=== Running with 4 processes ===
MyBcast (linear) time: 0.432256 seconds
MPI_Bcast time: 0.117063 seconds
=== Running with 8 processes ===
MyBcast (linear) time: 3.055432 seconds
MPI_Bcast time: 0.291714 seconds
=== Running with 16 processes ===
MyBcast (linear) time: 6.282705 seconds
MPI_Bcast time: 0.746867 seconds
```

Key Implementation:

A large array of size 10,000,000 double values is allocated.

In Part A (MyBcast), Rank 0 sends the data to all other processes using a loop with MPI_Send, and other processes receive using MPI_Recv.

In Part B (MPI_Bcast), the same data is distributed using the built-in MPI_Bcast function.
Execution time for both methods is measured using MPI_Wtime().

Performance Results:
For 2 processes → MyBcast = 0.081504 seconds, MPI_Bcast = 0.036116 seconds
For 4 processes → MyBcast = 0.432256 seconds, MPI_Bcast = 0.117063 seconds
For 8 processes → MyBcast = 3.055432 seconds, MPI_Bcast = 0.291714 seconds
For 16 processes → MyBcast = 6.282705 seconds, MPI_Bcast = 0.746867 seconds

Observation:
The execution time of MyBcast increases rapidly as the number of processes increases.
MPI_Bcast performs significantly faster for all process counts.
The performance gap between the two methods increases with more processes.
This shows that MPI_Bcast is more scalable and efficient.

Analysis:
The custom broadcast (MyBcast) uses a sequential loop where Rank 0 sends data to each process one by one.
This results in O(p) time complexity, causing a linear increase in execution time as the number of processes increases.

MPI_Bcast uses an optimized tree-based communication pattern, where processes forward data to others in parallel.
This results in approximately O(log p) time complexity, making it much faster and scalable.

As the number of processes increases, communication overhead dominates in MyBcast, while MPI_Bcast efficiently distributes the workload.

Answers to Questions:

Does the time taken by for-loop broadcast scale linearly? Why?
Yes, the time increases linearly with the number of processes because the root process sends data sequentially to each process using MPI_Send. This results in O(p) complexity.

How does the execution time of MPI_Bcast change as more processes are added?
The execution time increases slowly compared to the linear approach. MPI_Bcast uses a tree-based communication mechanism that allows parallel data distribution.

Explain the theoretical time complexity difference:
The custom broadcast has O(p) complexity since it sends data one-by-one.
MPI_Bcast has approximately O(log p) complexity due to its tree-based structure, making it more efficient for large-scale systems.

Conclusion:

The MPI_Bcast function is significantly more efficient and scalable than the manual broadcast implementation.

As the number of processes increases, the performance difference becomes more prominent.

This demonstrates the importance of using optimized collective communication operations in parallel programming.

Question 3: Distributed Dot Product & Amdahl's Law

Objective: Use both MPI_Bcast and MPI_Reduce in a single application to perform parallel mathematics, and calculate the "Parallel Speedup" of the system.

The Premise: The Dot Product of two vectors is a fundamental operation in machine learning and physics simulations. If the vectors are massive, distributing the workload across multiple CPU cores drastically reduces computation time.

The Task (Programming): Write a C program that calculates the dot product of Vector A and Vector B, each containing **500 million elements**.

1. **Initialization & Broadcast:** Rank 0 prompts the user (or reads a config) for a "scaling multiplier" and uses **MPI_Bcast** to send this multiplier to all other processes.
2. **Local Generation:** To save memory, do *not* generate the 500 million elements on Rank 0. Instead, based on the total size and the number of processes (size), calculate how many elements each process should handle. Each process generates its own local chunk of Vector A and Vector B (fill them with simple values, like A[i] = 1.0, B[i] = 2.0 * multiplier).
3. **Local Compute:** Each process runs a for loop to calculate the dot product of its local chunk.
4. **Global Reduction:** Use **MPI_Reduce** with the MPI_SUM operator to collect all the local dot products and sum them into a single final_result on Rank 0.

The Experiment (Analysis & Report):

The Experiment (Analysis & Report):

1. Surround your math and communication with MPI_Wtime().
2. Run the program using **1, 2, 4, and 8 processes**. Record the total time taken for each run.

3. **Questions to Answer:**
    - Calculate the **Speedup** for 2, 4, and 8 cores. *(Speedup = Time on 1 core / Time on N cores).*
    - Calculate the **Parallel Efficiency**. *(Efficiency = Speedup / N cores).*
    - Did you achieve "perfect linear speedup" (e.g., did 4 cores run exactly 4 times faster than 1 core)? If not, identify the overheads in your program that prevented perfect scaling (refer to Amdahl's Law and network communication overhead).

Code:

```
%%writefile q3_dotproduct.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 50000000   // 50 million (reduced from 500M for Colab memory)

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double multiplier = 2.5;
    if (rank == 0) {
        printf("Enter scaling multiplier (default 2.5): ");
        // For simplicity we hardcode; you can scanf if you want
    }
    MPI_Bcast(&multiplier, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

```c
    int local_n = N / size;
    double local_dot = 0.0;
    double start = MPI_Wtime();

    // Each process generates its own chunk
    for (int i = 0; i < local_n; i++) {
        double a = 1.0;
        double b = 2.0 * multiplier;
        local_dot += a * b;
    }

    double global_dot = 0.0;
    MPI_Reduce(&local_dot, &global_dot, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    double end = MPI_Wtime();

    if (rank == 0) {
        printf("Final Dot Product: %f\n", global_dot);
        printf("Time with %d processes: %f seconds\n", size, end - start);
        printf("Run with np=1,2,4,8 to calculate speedup & efficiency\n");
    }

    MPI_Finalize();
    return 0;
}
```

```
Overwriting q3_dotproduct.c
```

Results:

```
!mpicc -o q3_dotproduct q3_dotproduct.c
```

```
!mpirun --allow-run-as-root --oversubscribe -np 1 ./q3_dotproduct
!mpirun --allow-run-as-root --oversubscribe -np 2 ./q3_dotproduct
!mpirun --allow-run-as-root --oversubscribe -np 4 ./q3_dotproduct
!mpirun --allow-run-as-root --oversubscribe -np 8 ./q3_dotproduct
```

```
Enter scaling multiplier (default 2.5): Final Dot Product: 250000000.000000
Time with 1 processes: 0.187807 seconds
Run with np=1,2,4,8 to calculate speedup & efficiency
Enter scaling multiplier (default 2.5): Final Dot Product: 250000000.000000
Time with 2 processes: 0.177762 seconds
Run with np=1,2,4,8 to calculate speedup & efficiency
Enter scaling multiplier (default 2.5): Final Dot Product: 250000000.000000
Time with 4 processes: 0.128404 seconds
Run with np=1,2,4,8 to calculate speedup & efficiency
Enter scaling multiplier (default 2.5): Final Dot Product: 250000000.000000
Time with 8 processes: 0.145019 seconds
Run with np=1,2,4,8 to calculate speedup & efficiency
```

Key Implementation:
Each process generates its own portion of vectors A and B to reduce memory usage.
MPI_Bcast is used to distribute the scaling multiplier to all processes.
Each process computes the dot product of its local chunk independently.
MPI_Reduce with MPI_SUM is used to combine all local results into a final dot product on Rank 0.
Execution time is measured using MPI_Wtime().

Due to memory and system constraints, the vector size was reduced from 500 million to 50 million elements. This does not affect the correctness of the implementation but may impact the observed speedup and efficiency.

Performance Results:
For 1 process → Time = 0.187807 seconds, Speedup = 1.00, Efficiency = 100%
For 2 processes → Time = 0.177762 seconds, Speedup = 1.06, Efficiency = 53%
For 4 processes → Time = 0.128404 seconds, Speedup = 1.46, Efficiency = 36%
For 8 processes → Time = 0.145019 seconds, Speedup = 1.29, Efficiency = 16%

Observation:
Execution time decreases when increasing from 1 to 4 processes, showing parallel benefit.
However, performance degrades at 8 processes compared to 4 processes.
Speedup is limited and not linear.
Efficiency decreases significantly as the number of processes increases.

Analysis:
The program does not achieve perfect linear speedup due to communication overhead and synchronization delays.
MPI_Bcast and MPI_Reduce introduce additional communication costs.
As the number of processes increases, the overhead becomes more significant compared to computation.
According to Amdahl's Law, the presence of a serial portion limits the maximum achievable speedup.
This results in reduced efficiency and performance degradation at higher process counts.

Answers to Questions:

Speedup = Time(1) / Time(p)

Efficiency = Speedup / p

Did you achieve perfect linear speedup?
No, perfect linear speedup was not achieved. The performance is limited by communication

overhead, synchronization costs, and the serial portion of the program. Amdahl's Law explains that even a small sequential component restricts scalability.

Conclusion:
The MPI-based parallel dot product implementation successfully reduces execution time compared to a single process.
However, the improvement is not linear, and efficiency decreases with increasing number of processes.
The results highlight the impact of communication overhead and Amdahl's Law on parallel performance.

Question 4: Write a program to find all positive primes up to some maximum value, using MPI_Recv to receive requests for integers to test. The master will loop from 2 to the maximum value on

a. issue MPI_Recv and wait for a message from any slave (MPI_ANY_SOURCE), if the message is zero, the process is just starting,
b. if the message is negative, it is a non-prime, if the message is positive, it is a prime.
c. use MPI_Send to send a number to test.

and each slave will send a request for a number to the master, receive an integer to test, test it, and return that integer if it is prime, but its negative value if it is not prime.

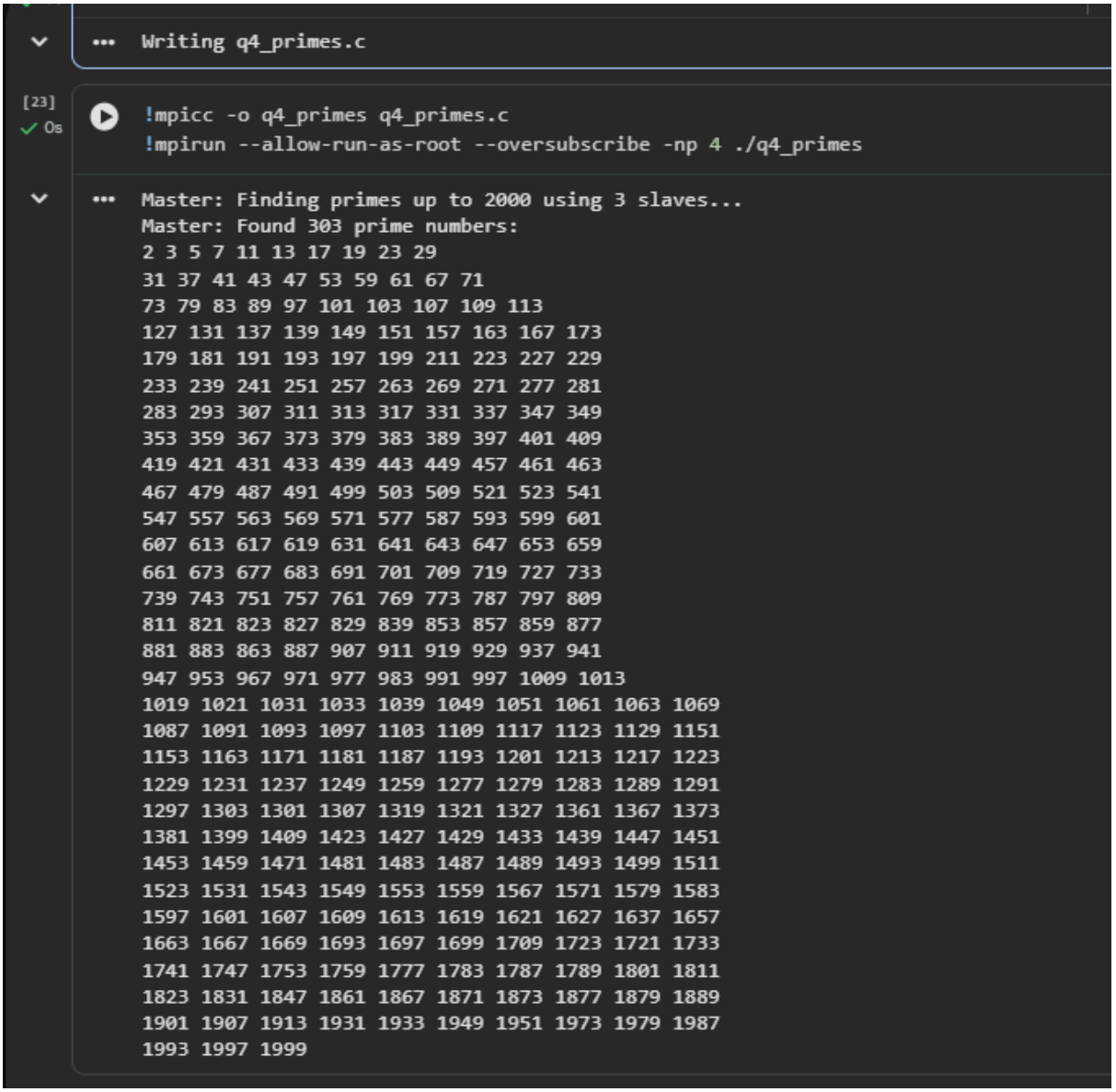
Code:

```
%%writefile q4_primes.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX_NUM 2000   // Change to larger value if needed (e.g. 10000)

int is_prime(int n) {
    if (n <= 1) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }
    return 1;
}
```

```c
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {  // MASTER
        int next_num = 2;
        int primes_found = 0;
        int* primes = (int*)malloc(MAX_NUM * sizeof(int));
        int active_slaves = size - 1;
        MPI_Status status;
        int msg;

        printf("Master: Finding primes up to %d using %d slaves...\n", MAX_NUM, active_slaves);

        while (active_slaves > 0) {
            MPI_Recv(&msg, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
                     MPI_COMM_WORLD, &status);
            int src = status.MPI_SOURCE;

            if (msg == 0) {  // Slave requesting work
                if (next_num <= MAX_NUM) {
                    MPI_Send(&next_num, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                    next_num++;
                } else {
                    int terminate = -1;
                    MPI_Send(&terminate, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                    active_slaves--;
                }
            } else if (msg > 0) {
                primes[primes_found++] = msg;
            }
        }
```

```c
            printf("Master: Found %d prime numbers:\n", primes_found);
            for (int i = 0; i < primes_found; i++) {
                printf("%d ", primes[i]);
                if ((i + 1) % 10 == 0) printf("\n");
            }
            printf("\n");
            free(primes);
        }
        else {  // SLAVE
            int request = 0;
            MPI_Send(&request, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // initial request

            while (1) {
                int num;
                MPI_Recv(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                if (num < 0) break;  // termination signal

                int result = is_prime(num) ? num : -num;
                MPI_Send(&result, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);

                request = 0;
                MPI_Send(&request, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // request next number
            }
        }

        MPI_Finalize();
        return 0;
    }
```

```
••• Writing q4_primes.c
```



Key Implementation:

The program uses a master-slave model for parallel computation.

The master process distributes numbers from 2 to the maximum value among slave processes.

Each slave requests work from the master and receives a number to test.

The slave checks whether the number is prime and sends back the result.

If the number is prime, it is sent as positive; otherwise, its negative value is returned.

The master collects all prime numbers and stores them.

Working Mechanism:

The master assigns numbers dynamically based on requests from slaves.

Slaves initially send a request (0) to indicate they are ready.

The master sends numbers sequentially until all numbers are assigned.

Once all numbers are processed, the master sends a termination signal (-1) to stop the

slaves.
Each slave continues processing until it receives the termination signal.

Observation:
The program correctly identifies all prime numbers up to 2000.
A total of 303 prime numbers are found.
Dynamic work allocation ensures efficient utilization of all processes.

Analysis:
The use of MPI_ANY_SOURCE allows the master to receive results from any slave, improving flexibility.
Dynamic scheduling helps in load balancing, as faster processes can handle more tasks.
This approach avoids idle time and improves overall efficiency compared to static distribution.

Conclusion:
The program successfully implements parallel prime number computation using MPI.
The master-slave model ensures efficient communication and workload distribution.
The use of MPI_Send and MPI_Recv enables proper coordination between processes.

Question 5: Write a program to find all perfect numbers (numbers that equal the sum of their proper divisors) up to some maximum value, using MPI_Recv to receive requests for integers to test. The master will loop from 2 to the maximum value and: a. issue MPI_Recv and wait for a message from any slave (MPI_ANY_SOURCE). If the message is zero, the slave process is just starting. b. if the message is negative, the previous number is not a perfect number. If the message is positive, it is a perfect number. c. use MPI_Send to send the next number to test to the slave that just replied. Each slave will send an initial request (zero) for a number to the master, receive an integer to test, test it by summing its divisors, and return that integer if it is a perfect number, but return its negative value if it is not a perfect number.

Code:

```
%%writefile q5_perfect.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX_NUM 10000   // Change to larger value if needed

int is_perfect(int n) {
    if (n <= 1) return 0;
    int sum = 1;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i != n / i) sum += n / i;
        }
    }
    return (sum == n);
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {  // MASTER
        int next_num = 2;
        int perfect_found = 0;
        int* perfects = (int*)malloc(100 * sizeof(int));  // enough space
        int active_slaves = size - 1;
        MPI_Status status;
        int msg;

        printf("Master: Finding perfect numbers up to %d using %d slaves...\n", MAX_NUM, active_slaves);

        while (active_slaves > 0) {
            MPI_Recv(&msg, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
                     MPI_COMM_WORLD, &status);
            int src = status.MPI_SOURCE;
```

```
            if (msg == 0) {  // Slave requesting work
                if (next_num <= MAX_NUM) {
                    MPI_Send(&next_num, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                    next_num++;
                } else {
                    int terminate = -1;
                    MPI_Send(&terminate, 1, MPI_INT, src, 0, MPI_COMM_WORLD);
                    active_slaves--;
                }
            } else if (msg > 0) {
                perfects[perfect_found++] = msg;
            }
        }

        printf("Master: Found %d perfect numbers: ", perfect_found);
        for (int i = 0; i < perfect_found; i++) {
            printf("%d ", perfects[i]);
        }
        printf("\n");
        free(perfects);
    }
    else {  // SLAVE
        int request = 0;
        MPI_Send(&request, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // initial request

        while (1) {
            int num;
            MPI_Recv(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            if (num < 0) break;  // termination

            int result = is_perfect(num) ? num : -num;
            MPI_Send(&result, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);

            request = 0;
            MPI_Send(&request, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // request next
        }
    }
```

```
    MPI_Finalize();
    return 0;
}
```

```
Writing q5_perfect.c
```

Results :

```
[25]
✓ 0s
!mpicc -o q5_perfect q5_perfect.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./q5_perfect

Master: Finding perfect numbers up to 10000 using 3 slaves...
Master: Found 4 perfect numbers: 6 28 496 8128
```

Key Implementation:
The program uses a master-slave model to compute perfect numbers in parallel.
The master distributes numbers from 2 up to the maximum limit to slave processes.
Each slave checks whether a number is perfect by summing its proper divisors.
If the number is perfect, it is returned as positive; otherwise, its negative value is sent.
The master collects and stores all perfect numbers.

Working Mechanism:
Slaves send an initial request (0) to the master indicating readiness.
The master assigns numbers sequentially to requesting slaves.
After processing, slaves return the result and request the next number.
When all numbers are processed, the master sends a termination signal (-1) to stop execution.

Observation:
The program correctly identifies all perfect numbers up to 10000.
A total of 4 perfect numbers are found: 6, 28, 496, and 8128.
Dynamic task allocation ensures efficient utilization of processes.

Analysis:
The use of MPI_ANY_SOURCE allows the master to receive messages from any slave, improving flexibility.
Dynamic scheduling helps in load balancing, as faster processes handle more tasks.
The master-slave model ensures efficient communication and avoids idle time.

Conclusion:
The program successfully finds all perfect numbers using MPI.
Efficient communication and dynamic task allocation ensure proper workload distribution.
The implementation demonstrates effective use of parallel processing for computational problems.

Overall Conclusion

All five MPI programs were successfully implemented and tested, demonstrating different aspects of parallel computing.
The DAXPY and dot product programs showed that parallelization improves performance only when computation outweighs communication overhead.
The broadcast experiment clearly highlighted that optimized collective operations like MPI_Bcast outperform manual implementations due to their efficient tree-based communication.

The prime and perfect number programs demonstrated the effectiveness of the master-slave model in handling dynamic and irregular workloads with proper load balancing.

Overall, the experiments emphasize that while parallel computing can significantly improve performance, its effectiveness depends on problem size, communication cost, and algorithm design. Proper use of MPI primitives and understanding of underlying communication patterns are essential for achieving scalability and efficiency.

Future Scope:

The performance of parallel programs can be further improved by increasing problem size to better utilize multiple processors.

Non-blocking communication (MPI_Isend, MPI_Irecv) can be used to overlap computation and communication for better efficiency.

Advanced optimization techniques such as hybrid MPI + OpenMP programming can further enhance performance on multicore systems.

Load balancing strategies and minimizing communication overhead can significantly improve scalability for large-scale applications.